

Week 2: Agentic coding with Rust

Jin-Guo Liu, Xi-Wei Pan

How can forces between atoms produce the motion and structure of a material? Build one two-dimensional molecular-dynamics (MD) program in Rust to investigate.

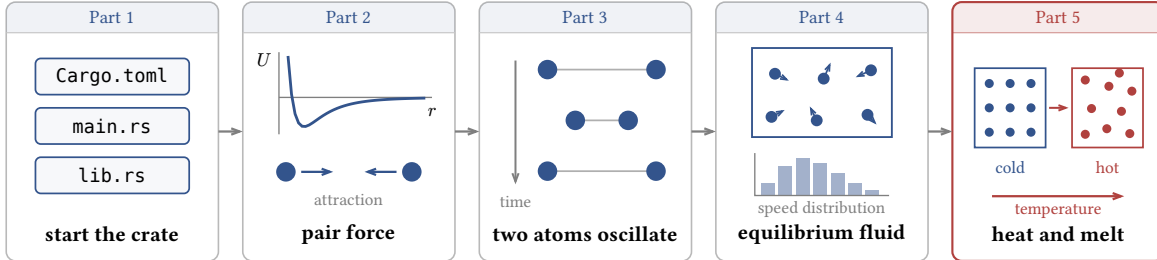


Figure 1: Task overview.

Goal: a tested `md` program in `week2/` of your existing `AMAT5315-2026Fall-Exercise` repository, a public heating animation, and a short recording of you running the check and reading the melt. Explain how its atoms move, how its Rust design organizes and shares their state, and what your measurements say makes it fast. The final checklist defines done.

Workspace: start the agent at your exercise repository root. All file paths below are relative to that root. Run grey command panels yourself in a second terminal. Send blue panels to the agent. The shell enters `week2/` in Part 1 and stays there until Part 5. The agent does not read this sheet; every blue panel carries all it needs. Keep the same program and its earlier checks throughout all five Parts.

Resources:

- [The Rust Book](#) each Part names its sections.
- [week2-sim.py](#) : the NumPy benchmark baseline.
- [week2-viewer.html](#) : the supplied viewer for a *trajectory*, the recorded positions and velocities over time.

Units: atom size σ , energy scale ε , mass m and Boltzmann's constant k_B are all 1. These are *reduced units*. Positions and velocities have two components.

Part 1: Rust ABC

Goal: a crate named `md` in `week2/` that builds, runs, and passes one test, and the reading skill to say what the compiler checks and what a test checks in the agent's Rust.

① READ

Rust's compiler turns source into a native executable with no runtime. *Cargo* builds and tests a project, called a *crate*. A *library* holds functions other code can call.

Our first model follows a particle in two dimensions, described by its position $\vec{x}$ and velocity $\vec{v}$. With no force, its velocity stays constant: this is *free flight*, implemented by `FreeFlight` below. $\vec{x}(t + \Delta t) = \vec{x}(t) + \vec{v}\Delta t$. Read the program below, following its state, update rule, and test.

```
struct System { // System owns two vectors.
    positions: Vec<f64; 2>, // f64 is a 64-bit floating-point number.
    velocities: Vec<f64; 2>, // 2 means two-dimensional vector.
}

impl System { // Methods on data: an object-oriented feature.
    // &self borrows for reading; usize is a size.
    fn n_atoms(&self) -> usize { self.positions.len() }
}

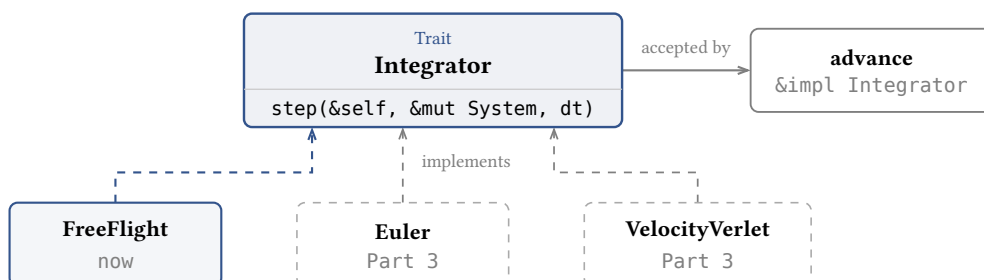
trait Integrator { // Every integrator must implement one time step function.
    fn step(&self, system: &mut System, dt: f64);
}

fn advance(method: &impl Integrator, system: &mut System, dt: f64) {
    method.step(system, dt); // Works with any declared implementation of Integrator.
}

struct FreeFlight; // One concrete implementation
impl Integrator for FreeFlight {
    fn step(&self, system: &mut System, dt: f64) {
        // iter_mut permits updates; zip pairs each position with one velocity.
        for (x, v) in system.positions.iter_mut().zip(&system.velocities) {
            x[0] += dt * v[0]; x[1] += dt * v[1];
        }
    }
}

#[test] // This attribute registers a function with cargo test.
fn free_particle() {
    let mut system = System { // mut allows the owned state to change.
        positions: vec![[0.0, 0.0]],
        velocities: vec![[1.0, -2.0]],
    };
    advance(&FreeFlight, &mut system, 0.5); // Use the shared driver.
    assert_eq!(system.positions[0], [0.5, -1.0]);
    assert_eq!(system.velocities[0], [1.0, -2.0]); // Velocity is unchanged.
}
```

A trait defines a shared contract. We may have different numerical update rules. `Integrator` requires each motion rule to provide a `step` method, so `advance` can use different rules through the same call.



Ownership and borrow checking make safe Rust memory-safe. These rules specify who is responsible for data and how other code may access it. Here, `System` owns its arrays and frees their storage when dropped. Functions borrow them without taking ownership. In the call above, `&FreeFlight` allows reading the rule, while `&mut system` allows exclusive access to modify the state.

The compiler checks before the program runs. Rust is *statically typed*: types are checked when the crate compiles. The compiler also checks that borrowed data stays alive and that access follows the borrowing rules. Its error messages point to the offending code and often suggest a fix. The agent can use this feedback to repair type and borrowing errors before running the program. A program that compiles has passed these checks; it has not been checked against the physics. That part is yours: choose the physical checks and inspect their results. This week, Rust runs the simulation's inner loop; Python supports exploration around it.

- Read [Rust Book, section 4.2](#). Explain why `&self` permits reading and `&mut System` permits updates.
- Skim [section 10.2](#). Find the trait, its required method, and its implementation in the example.
- Skim [section 11.1](#). Find how `#[test]`, `assert!`, and `cargo test` work together.

2 THINK

In `free_particle`, the particle starts at $\vec{x} = (0, 0)$ with velocity $\vec{v} = (1, -2)$. Calculate its position after $\Delta t = 0.5$ using [Equation 0](#) and compare your result with the values in `assert_eq!`. Then decide what would go wrong if `step` took `&System` instead of `&mut System`, and if the update wrote `x[0] += dt * v[1]`. One is a compiler error; the other only a test can catch.

The plan: the code built in this Part is only Hello World, with a library, an executable, and a test.

3 DO

Build a Hello World crate to practise compiling, running, and testing a Rust program. Reuse the `AMAT5315-2026Fall-Exercise` repository from week 1; do not create a new one. Pull it, then start the agent at its root. Windows users work inside WSL.



Set up Rust and create a Hello World project named `md` in `week2/`. Put the greeting in a library function, and add one test that checks it. Make cargo run print Hello, world! Run the test and commit.

4 VERIFY

In a second terminal, start at the exercise repository root.

(1) **The crate builds, runs, and passes its test.**



```
cd week2/md
cargo run
cargo test --lib
```

Expected: the program prints `Hello, world!`; the test run ends with `1 passed; 0 failed`.

(2) **Each kind of code has one place.** Cargo laid the crate out this way:

<code>Cargo.toml</code>	the crate's name and dependencies
<code>src/lib.rs</code>	the library: physics that the tests and the executable both call
<code>src/main.rs</code>	the executable: reads the command line, calls the library

Read the test in `lib.rs` and confirm it calls the same greeting function as the executable.

(3) Explain ownership and the two kinds of borrow in the reading example.

Return to `week2/` for the following Parts:



```
cd ..
```

Cargo commands there use `--manifest-path md/Cargo.toml` to select the crate. Output paths remain relative to `week2/`.

Part 2: From pair energy to a tested force

Goal: the Lennard-Jones energy and force in the library, a test that the force is minus the slope of the energy, and a field plot with arrows outward inside r_0 and inward outside.

1 READ

Molecular dynamics moves atoms by Newton’s second law: the acceleration of each atom is the total force on it. This week the atoms are neutral, like argon. Two such atoms attract weakly at a distance and repel strongly once their electron clouds overlap. A *pair potential* $U(r)$ encodes both effects as one energy that depends only on the separation r , the distance between atom centres. Every force in the simulator comes from this one function. This Part builds it and its derivative in Rust, and tests the derivative against the potential.

The model. The attraction is the *van der Waals* interaction between induced dipoles, whose energy falls off as $-r^{-6}$. The repulsion has no equally simple law; Lennard-Jones took r^{-12} , the square of the attraction term, for convenience. Their difference is the *Lennard-Jones potential*:

$$U(r) = 4[r^{-12} - r^{-6}]. \quad (1)$$

The energy crosses zero at $r = 1$. The factor 4 sets the well depth to 1. Setting $U'(r) = 0$ gives the minimum at $r_0 = 2^{1/6} \approx 1.122$, where $U(r_0) = -1$.

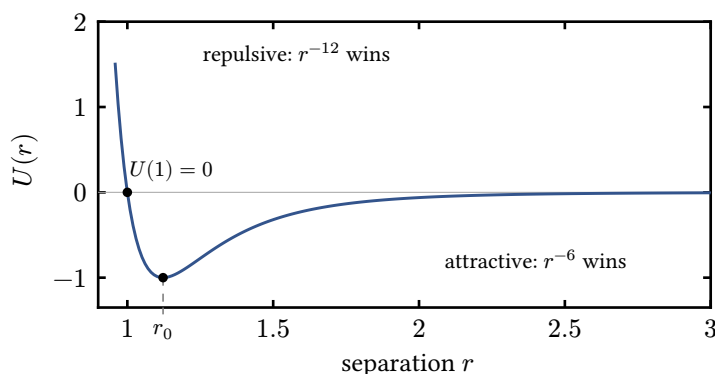


Figure 2: Lennard-Jones potential energy $U(r)$ between two atoms as a function of their separation r , in reduced units.

A unit test of the energy the agent will write looks like this:

```
#[test]
fn well_depth() {
    let r0 = 2.0_f64.powf(1.0 / 6.0);
    assert!((energy(r0) + 1.0).abs() < 1e-12);
}
```

It evaluates the energy at its minimum and checks the well depth. A placeholder body *panics* when called, so a test like this fails until `energy` is implemented.

A *force* is minus the slope of the potential. Differentiate U by hand once:

$$F(r) = -\frac{dU}{dr} = \frac{24}{r}[2r^{-12} - r^{-6}], \quad (2)$$

The scalar $F(r)$ is positive when the atoms repel and zero at r_0 . The force on atom i from atom j is exactly minus the force on j from i . This is Newton’s third law.

Testing a derivative. The *central difference* $-\frac{U(r+h)-U(r-h)}{2h}$ estimates $F(r)$ from the energies at distances $r-h$ and $r+h$. Its error shrinks like h^2 until rounding becomes significant. For example, choose $h = 10^{-5}$ and compute this estimate from `energy`. Compare it with `force(r)`, which evaluates the force formula directly. Allow an absolute tolerance of $10^{-6} \times \max(1, |F(r)|)$ to account for numerical error. A larger difference means the check fails.

- Read chapter 4, “Molecular Dynamics Simulations,” in Daan Frenkel and Berend Smit’s *Understanding Molecular Simulation: From Algorithms to Applications*, third edition, 2023 [1]. Focus on how a pair potential determines atomic motion. Keep this as the week’s main MD textbook.

② THINK

Fix atom i at the origin and put atom j anywhere in the plane. From the sign and the zero of $F(r)$, which way does j move when its distance from i is less than r_0 , which way when it is greater than r_0 , and on which circle does the force vanish?

Now suppose the agent writes the force with the wrong sign or a wrong coefficient: the program still compiles and runs, and the atoms still move, only wrongly. How to design tests to reject this?

The defence is to test the force against the energy: a numerical derivative of `energy` compares two separately implemented functions, the well-depth test pins their common scale, and the field plot lets you see the direction of the forces you just worked out.

The plan: extend the md crate with those tests first, have the agent implement the energy and then the force until they pass, and plot the finished functions to compare with the reference in `VERIFY`.

③ DO

Two prompts: first the tests, then the code. Start the agent at the exercise repository root; your terminal stays inside `week2/`.

(1) Write the tests, and watch them fail:

💬 Add unit tests to the md crate in `week2/md/` for the Lennard-Jones pair energy and its force, in reduced units. Request: both tests fail until the functions exist. Show me the failing tests and commit.

Expected: the two new tests fail because the functions are unimplemented; the Hello World test still passes. Testers call this state *red*.

(2) Make them pass:

💬 Implement the energy, then the force, and commit after each once its tests pass. Request: keep the tests unchanged.

Open the energy function and compare it with [Equation 1](#). Open the module containing `force`: it must be its own formula, not a difference of `energy` calls, or the test compares a thing with itself.

④ VERIFY

Each check confirms one feature of the crate. Run the commands yourself in a second terminal window inside `week2/`, not through the agent: a check outsourced to the thing being checked proves nothing.

(1) **The tests pass.**

```
> cargo test --manifest-path md/Cargo.toml
```

Expected: all tests pass.

(2) Open and review the force test.

Expected: It must compare the force with the energy, and a numerical derivative of the energy is the usual way. If it only checks values the agent computed by hand, **ask the agent for the derivative comparison**. Read the separations, the step h , and the tolerance it chose; the separations should straddle r_0 , and the numbers should agree with **Testing a derivative**.

(3) **The force field has the right shape.** Have the agent draw it from your functions:

💬 Plot the pair energy as colors and the force as arrows around one atom, using the energy and force functions of the md crate in `week2/md/`. Request: save it as `week2/field.png` with the command that generates it, and commit.

Open `week2/field.png` and compare with this reference. Pass: outward arrows inside r_0 , inward arrows outside, and a ring of negative energy around the repulsive core. An inverted sign or a missing ring means a wrong formula that the tests may not catch.

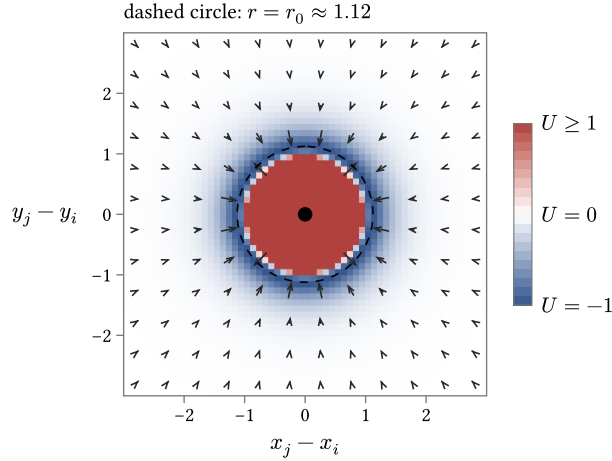


Figure 3: Pair energy and force around a fixed atom: colour shows potential energy (capped at $U = 1$); arrows show force (lengths compressed). The dashed circle marks $r = r_0$.

Part 3: From force to motion

Goal: a simulator that moves two atoms with either Euler or velocity-Verlet through one trait, and a test from the same initial state showing Euler's energy drifting while Verlet's stays bounded.

① READ

We now want to predict how two atoms move under their mutual force. Newton's second law relates that force to acceleration; we use it to update their velocities and positions in small time steps. This procedure is *numerical integration*. Even with the correct force, errors in these updates can accumulate over thousands of steps. We will compare two update rules on an isolated pair of atoms, called a *dimer*, using the same Rust trait in Part 1's example.

Vector forces. Define $\vec{d} = \vec{x}_i - \vec{x}_j$ and $r = |\vec{d}|$. The forces are $\vec{F}_i = F(r)\frac{\vec{d}}{r}$ and $\vec{F}_j = -\vec{F}_i$. Since $m = 1$, acceleration equals force.

An *integrator* advances positions and velocities in steps of duration Δt . Forward Euler uses the old state for both updates:

$$\vec{x}_{n+1} = \vec{x}_n + \vec{v}_n \Delta t, \quad \vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \Delta t. \quad (3)$$

Velocity-Verlet uses the force at both ends of the step:

```

1: function VELOCITY-VERLET-STEP( $\vec{x}_n, \vec{v}_n, \vec{a}_n, \Delta t$ )
2:    $\vec{v}_{n+1/2} \leftarrow \vec{v}_n + \frac{\Delta t}{2} \vec{a}_n$     ▷ half kick with the old accelerations
3:    $\vec{x}_{n+1} \leftarrow \vec{x}_n + \Delta t \vec{v}_{n+1/2}$   ▷ drift with the half-step velocities
4:    $\vec{a}_{n+1} \leftarrow \text{accelerations}(\vec{x}_{n+1})$   ▷ the step's one force evaluation
5:    $\vec{v}_{n+1} \leftarrow \vec{v}_{n+1/2} + \frac{\Delta t}{2} \vec{a}_{n+1}$   ▷ half kick with the new accelerations
6:   return  $\vec{x}_{n+1}, \vec{v}_{n+1}, \vec{a}_{n+1}$     ▷ keep  $\vec{a}_{n+1}$  for the next step
7: end
```

Before the first step, call `accelerations` to compute the initial accelerations from the atom positions using $\vec{a} = \frac{\vec{F}}{m}$. Each later step needs one new force evaluation, and the returned $\vec{a}_{n+1}$ is the next step's input.

What should stay constant? An isolated system exchanges kinetic and potential energy. We can test the implementation using this property. Their sum is $E = E_{\text{kin}} + E_{\text{pot}}$, with $E_{\text{kin}} = \frac{1}{2} \sum_i |\vec{v}_i|^2$ and $E_{\text{pot}} = \sum_{i < j} U(r_{ij})$. Record the relative energy error $(E(t) - E_0)/|E_0|$, where E_0 is the initial energy. A persistent trend in this error is *energy drift*.

Why compare these methods? For a fixed final time, an order- p method's solution error scales as Δt^p once the timestep is sufficiently small. Euler is first order; velocity-Verlet is second order. Velocity-Verlet is also *symplectic*: it preserves the geometric structure of Hamiltonian motion. For a smooth potential and a sufficiently small timestep, its energy error typically oscillates instead of drifting steadily [2]. Reducing the timestep improves Euler's accuracy over a fixed interval, but it does not give Euler this long-time conservation property.

- Continue chapter 4 of Frenkel and Smit. Trace the order of the Verlet updates and identify which force evaluation uses the new positions.
- Revisit the trait example in [Part 1](#). Explain how one driver can use two different update rules.
- Read the [superpowers README](#) and its `brainstorming`, `writing-plans`, and `test-driven-development` skills. Identify what the agent should show before implementing. An agent without these skills can use *plan mode*, where it proposes steps before editing.

② THINK

The goal is a simulator whose update rule keeps an isolated system's energy constant. The smallest system that can tell the two update rules apart is one bond: two atoms released from rest at separation 1.2, where the force pulls them together.

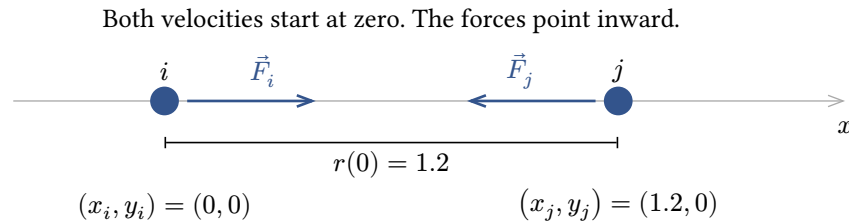


Figure 4: The dimer experiment. Use this initial state into both integrators.

Given	Value
Model	Two atoms in 2D, $m = 1$, open boundaries, plain Lennard-Jones potential.
Run	$\Delta t = 0.01$, 500 steps for both methods; extend Verlet to 5000 steps.
Measurement	$E(t) = \frac{1}{2} \sum_i \vec{v}_i ^2 + U(r(t)); E_0 = U(1.2).$

Use one test driver through a shared trait, with the same initial state and timestep, so only the update rule changes. Record the relative energy error at every step. The ten times longer Verlet run tests whether its energy error stays bounded. On the 500-step run, require Verlet's maximum relative error to stay below 10^{-3} and Euler's final error to exceed 0.5.

The plan: have the agent design the simulator around that interface, plan with the dimer test first, and implement until the test is green. Then plot both error curves from your own integrators and compare with the reference in `VERIFY`.

③ DO

- (1) Install the planning skills. Start the agent at the exercise repository root:

Install the brainstorming, writing-plans, test-driven-development, and requesting-code-review skills from <https://github.com/obra/superpowers> into this repository's skills folder. Commit.

- (2) Brainstorm, plan, and build. Start a fresh session so the new skills load:

Brainstorm with me to extend the `md crate` in `week2/md/` into a two-atom molecular dynamics simulation. Request: forward Euler and velocity-Verlet integrators sharing one Rust trait, so the same experiment runs with either, and the total-energy error over time as the experiment's output.

Before it starts, tell it:

- the initial state and run from the **Given** table above;
- the design in Part 1's example;
- what counts as conserving: Verlet's maximum error below 10^{-3} and Euler's final error above 0.5 on the same initial state.

The skills lead from here: the agent asks about the design and saves it under `docs/superpowers/`, offers to write a step-by-step plan, then executes the plan test-first. Answer its questions and approve each stage. If it offers to skip the design because the task is small, say no. Read the plan when it appears: the test must come

before any implementation step. Expected: a red run where the dimer test fails or does not compile yet, committed, then a green run.

④ VERIFY

Same rule, same window: you run the checks, not the agent.

(1) Both integrators face the same test.

```
> cargo test --manifest-path md/Cargo.toml
```

Expected: all green, and the suite contains a dimer test that runs both integrators from the same initial state. Open the dimer test and confirm that the only difference between the two runs is which integrator value is passed. If the test calls two differently named step functions, the trait is decoration; ask the agent to route both through the trait.

(2) Verlet conserves energy and Euler does not. Have the agent plot it from your own integrators:

💬 Add an example to the md crate in week2/md/ that plots the relative energy error of the two-atom simulation against time: left panel, both integrators for 500 steps at $dt = 0.01$; right panel, velocity-Verlet alone for 5000 steps with the error scaled by 1000. Request: save it as week2/dimer.png and record the command that reproduces it.

Run the example and commit it with the image.

Open `week2/dimer.png` and compare with the left panel of this reference, then inspect the long Verlet run in the right panel:

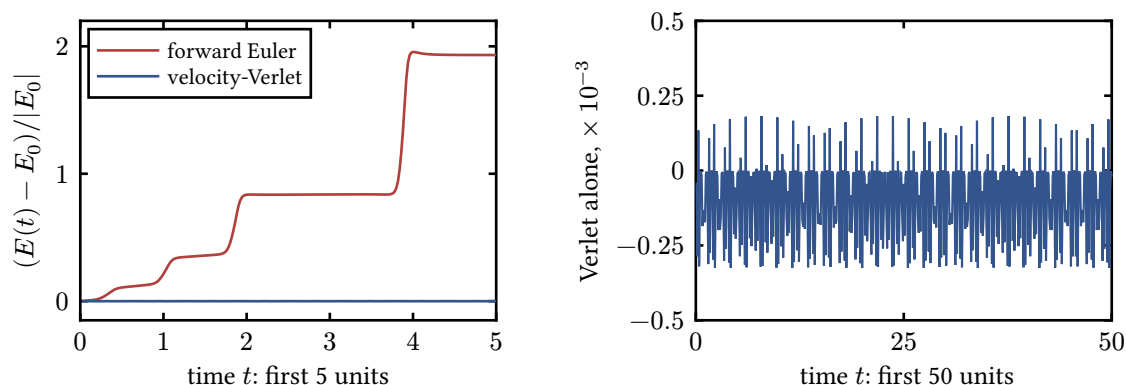


Figure 5: Left: Euler's relative energy error reaches 1.9 at $t = 5$. Right: Verlet's stays within $\pm 3 \times 10^{-4}$ without drift over ten times longer (vertical scale $\times 1000$).

(3) Explain your simulator. Find the velocity-Verlet step and trace its updates against the algorithm in READ. Locate the struct that owns the particle arrays, a shared borrow, and a mutable borrow. Explain why each function needs that access. Find the trait and both implementations; names may differ from the examples.

Extension: time reversal. Velocity-Verlet is reversible: run n steps, negate all velocities, run n more, and the atoms return to their starting positions up to rounding. Write that as a test with $n = 200$. Then try it on `Euler` and explain where the asymmetry enters.

Part 4: From two atoms to an equilibrium fluid

Goal: `md run` recording an $N = 100$ equilibrium fluid, `md check` reporting `PASS` on energy drift, temperature, and the speed distribution recomputed from the raw frames, `md video` animating the run, and `make reproduce` regenerating it from a fresh clone.

① READ

Statistical mechanics connects particle motion with measurable averages such as temperature. The dimer tested an update rule; a many-atom run will test whether the simulation reproduces equilibrium statistics. Add a periodic box and prepare a fluid at a target temperature. Record its motion so we can check the physics from the saved trajectory.

The lattice. A *triangular lattice* has staggered rows with six nearest neighbours around each interior atom. The *density* ρ counts atoms per unit area. Use $N = 100$ atoms at $\rho = 0.8$ for the equilibrium experiment. The rectangular box has horizontal width L_x and vertical height L_y .

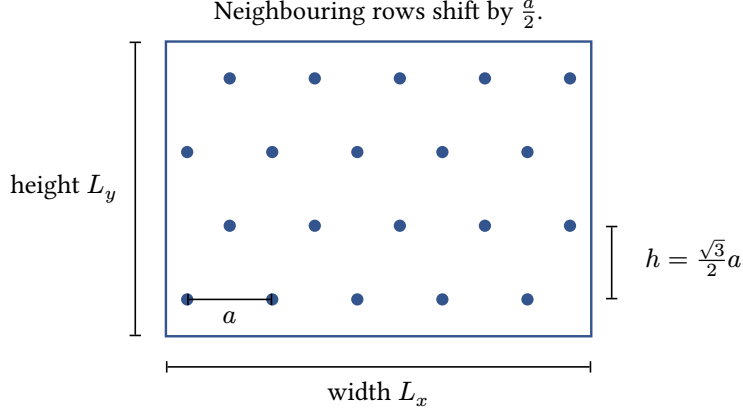


Figure 6: Lattice geometry, shown with fewer atoms for legibility. Each atom occupies area ah .

Calculate the box size. With 10 columns and 10 rows, the box dimensions are $L_x = 10a$ and $L_y = 10h$. From $\rho = \frac{1}{ah}$ and $h = \frac{\sqrt{3}}{2}a$,

$$a = \sqrt{\frac{2}{\sqrt{3}\rho}} \approx 1.2014, \quad L_x = 10a \approx 12.0141, \quad L_y = 10h \approx 10.4045. \quad (4)$$

The initial coordinates are $x_{ij} = (i + \frac{1}{2}(j \bmod 2))a$ and $y_{ij} = jh$, with $i, j = 0, \dots, 9$.

For a different atom count N at the same density, keep a and h unchanged. Arrange the atoms in $\sqrt{N}$ rows and $\sqrt{N}$ columns, so the box dimensions become $L_x = \sqrt{N}a$ and $L_y = \sqrt{N}h$. For example, the runs with 400 and 1600 atoms use 20×20 and 40×40 grids, respectively.

Periodic boundaries. To represent the interior of a material, imagine repeating the box in both directions. These are *periodic boundary conditions*. An atom leaving one edge re-enters at the opposite edge with the same velocity. Atoms near opposite edges can therefore be close neighbours. Compute their separation using the nearest periodic copy, called the *minimum image*.

Use an even number of rows so the alternating row offsets continue across the top and bottom periodic boundary. The 10-, 20-, and 40-row grids all meet this requirement.

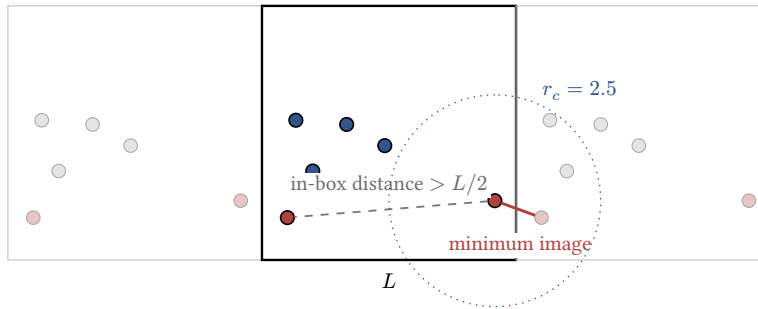


Figure 7: Periodic box with grey periodic copies. Red line: nearest-image separation. Dotted circle: cutoff radius $r_c = 2.5 < \frac{L}{2}$.

For each axis, let d be the coordinate difference and L the box length (L_x horizontally, L_y vertically). The nearest-image difference is $d - L \cdot \text{round}(\frac{d}{L})$. After each step, wrap positions back to $0 \leq x < L_x$ and $0 \leq y < L_y$ by adding or subtracting whole box lengths, keeping velocities unchanged.

The cutoff. Pairs with nearest-image separation $r \geq r_c$ do not interact. The cutoff radius $r_c = 2.5$ is **less than half either box dimension**, so at most one periodic copy of each neighbour lies within range. The *shifted potential* subtracts a constant inside this cutoff:

$$U_{\text{cut}(r)} = \begin{cases} U(r) - U(r_c), & r < r_c \\ 0, & r \geq r_c \end{cases}. \quad (5)$$

It approaches zero continuously. Its force is Part 2's force for $r < r_c$ and zero outside, so the force has a small jump. This breaks the smooth-potential assumption behind Part 3's energy argument. Measure the drift to check this modified model. Keep the plain potential and open boundaries available for the isolated dimer test.

Preparing the run. *Equilibration* lets the initial arrangement relax at the target temperature. *Production* is the later interval used for measurements. A *thermostat* changes velocities to bring the kinetic temperature toward a target. This week's thermostat rescales velocities. Draw independent Gaussian components of mean zero and variance $T = 0.5$ with seed 2026. Subtract the mean velocity to stop centre-of-mass motion. Compute the kinetic energy and rescale every component by $\sqrt{\frac{T_{\text{target}}}{T_{\text{thermo}}}}$. The thermostat temperature is $T_{\text{thermo}} = \frac{2E_{\text{kin}}}{2N-2}$: removing the mean velocity removes two independent components. Rescale initially and every 50 equilibration steps. Production time starts at zero after equilibration.

The md command. A *command-line interface* (CLI) is a program driven by flags such as `--n 100`. The simulator becomes one, with three subcommands. `md run` simulates and writes files into `--out`; `md check` reads those files and recomputes the physics without advancing the simulation; `md video` renders them as a short MP4, the atoms beside the pair-structure plot $g(r)$ defined under **Pair structure** below (install `ffmpeg` first; the program may call it to encode the frames). Every `run` flag defaults to the value of the *contract run*, the settings every later comparison uses, so the first two commands below are equal:

```
md run --n 100 --rho 0.8 --temperature 0.5 --dt 0.01 \
      --eq-steps 2000 --steps 10000 --sample-every 50 --seed 2026 --out artifacts
md run --out artifacts
md check artifacts # each value beside its limit, PASS or FAIL
md video artifacts --out artifacts/run.mp4 # one frame per saved frame, under 2 MB
```

`md check` exits with an error when any check fails or the files are malformed. The files use *JSON*, a plain-text format for structured data; a *frame* is one saved state. A *Makefile* gives commands names: `make reproduce` will run the default run, so the instructor's gate needs no instructions.

File in <code>--out</code>	Required fields
<code>run.json</code>	<code>n</code> , <code>rho</code> , <code>box</code> as <code>[Lx, Ly]</code> , <code>dt</code> , <code>temperature</code> , <code>eq_steps</code> , <code>steps</code> , <code>sample_every</code> , <code>seed</code> , <code>integrator</code> as <code>"velocity-verlet"</code> .
<code>traj.jsonl</code>	One JSON object per saved production frame: <code>step</code> , <code>t</code> equal to <code>step * dt</code> , <code>pos</code> wrapped into the box, <code>vel</code> , <code>E_pot</code> from the shifted potential, and <code>E_kin</code> .

Save each production step divisible by `sample_every`, excluding step zero. Thus 1000 steps sampled every 10 give exactly 100 frames.

Speeds. Pool speeds $v = |\vec{v}|$ from the saved frames. Their mean squared speed estimates $T_{\text{speed}} = \frac{\langle v^2 \rangle}{2}$. The two-dimensional *Maxwell-Boltzmann* speed density at that temperature is

$$f(v) = \frac{v}{T_{\text{speed}}} \exp\left(-\frac{v^2}{2T_{\text{speed}}}\right). \quad (6)$$

The shape statistic. Sort the pooled speeds into 24 bins with equal predicted probability. Define their edges by $b_k = \sqrt{-2T_{\text{speed}} \ln(1 - \frac{k}{24})}$ for $k = 0, \dots, 23$, with $b_{24} = \infty$. For M pooled speeds, the expected count in each bin is $E_b = \frac{M}{24}$. Let O_b be its observed count. The statistic is

$$\frac{\chi^2}{22} = \frac{1}{22} \sum_b \frac{(O_b - E_b)^2}{E_b}. \quad (7)$$

The denominator is $24 - 1 - 1$: one constraint fixes the total count, and one parameter estimates temperature.

Pair structure. The *radial distribution function* $g(r)$ counts neighbours in rings around an atom. Divide by the count expected at uniform density, then average over atoms and frames.

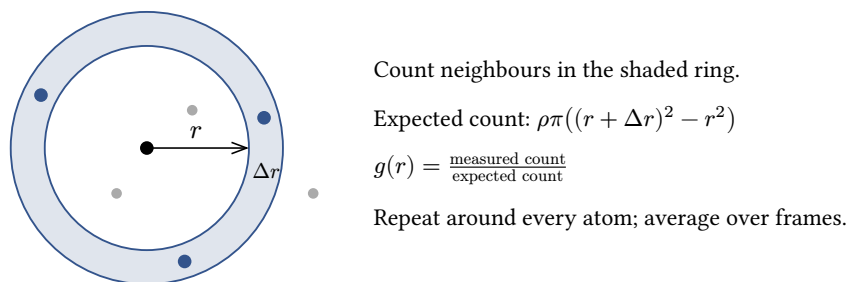


Figure 8: A counting-ring schematic. Use distances up to half the shorter box side. Repeated peaks indicate neighbour shells; Part 5 compares cold and hot runs.

The viewer's *long-range contrast* is the root-mean-square value of $g(r) - 1$ over its plotted bins with $r > 2$.

What must a correct run satisfy? The check recomputes everything from positions and velocities and only cross-checks the stored energies. Let E_0 be the energy of the **first saved frame** and $k = \max(1, \lfloor \frac{\text{frame count}}{10} \rfloor)$. Compare the mean energies of the first and last k frames:

Check	Measured quantity	Pass condition
Secular drift	$\frac{ \bar{E}_{\text{last } k} - \bar{E}_{\text{first } k} }{ E_0 }$	$< 2 \times 10^{-3}$
Temperature	$ T_{\text{speed}} - 0.5 $	< 0.05
Speed shape	$\frac{\chi^2}{22}$ with the 24 bins above	< 2

Secular drift measures the net shift between early and late energy averages. Together these checks test energy conservation, mean temperature, and speed shape. The speed-shape bound is a practical tolerance for this run, whose saved samples are correlated; it is not a formal statistical significance test.

- Read chapter 5, “Computer Experiments,” in Frenkel and Smit. Focus on preparing a simulation and interpreting the measured results.
- Read about [periodic boundaries](#) and the [radial distribution function](#). Explain what the repeated box and the neighbour-shell peaks represent.
- Skim the [Maxwell-Boltzmann distribution](#) through the speed distribution. Use the 2D formula printed in this sheet for this run.
- Open the supplied [viewer](#). Find where to load the data; the plots stay empty until the simulation writes it.

2 THINK

The goal is a run whose saved frames pass three physics checks recomputed from the raw positions and velocities. The tested Verlet integrator and pair force stay; the box and cutoff only change which pairs interact, and Newton's third law still requires the total internal force to vanish. Random initial velocities do not put the lattice in equilibrium, so relaxation and measurement are separate stages, and only the unheated stage can test energy conservation:

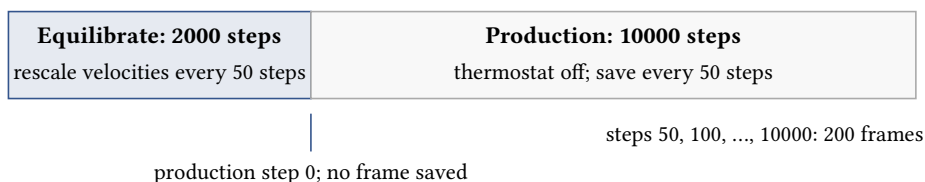


Figure 9: Equilibration: thermostat on. Production: thermostat off; check energy conservation here.

Total energy cannot stay constant while velocities are rescaled, so the drift check uses production frames only. The thermostat set T_{thermo} ; the saved frames yield T_{speed} , and for the same data $T_{\text{thermo}} = \frac{N}{N-1} T_{\text{speed}}$, about one percent apart at $N = 100$. A fluid in equilibrium has more than the right mean temperature: its whole speed histogram follows the Maxwell-Boltzmann curve at the measured T_{speed} , with no further adjustable parameter:

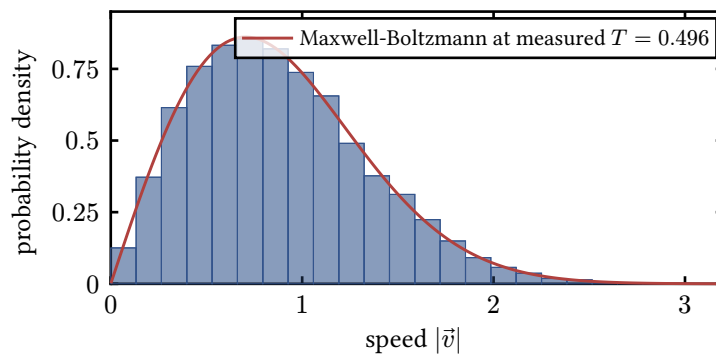


Figure 10: Speed histogram of this week’s contract run ($N = 100$, velocity-Verlet, 10^4 production steps) against the Maxwell-Boltzmann curve at the *measured* temperature. Estimating T from the mean squared speed fixes the curve without further adjustable parameters. This is the two-dimensional counterpart of Maxwell’s 1860 speed distribution [3].

A histogram can look plausible while missing its tails, so the shape statistic joins the plot. Energy drift answers a separate question, whether integration adds or removes energy, and the pair-structure view records the spatial order that the heating run will change.

The plan: have the agent design the run and the check, plan with four tests first (the total force vanishes, the cutoff is continuous, the contract run passes the physics bounds, and the binary writes frames), and implement. Add the reproduce rule so the run can be repeated from a fresh clone. Run the contract and look at the fluid in the viewer.

③ DO

(1) Brainstorm, plan, and build:

💬 Brainstorm with me to extend the Rust crate in `week2/md/` into a CLI tool for molecular dynamics simulation of a Lennard-Jones fluid. Request: run a simulation and save the trajectory, check the physics of a saved trajectory, and record a video of the molecular movements with the radial distribution function.

Before it starts, tell it, from READ:

- the start: a 10×10 triangular lattice at $\rho = 0.8$ (**The lattice**);
- the box: periodic, minimum image, shifted cutoff at $r_c = 2.5$ (**Periodic boundaries, The cutoff**);
- the stages: Gaussian velocities at T from the seed, rescaling every 50 equilibration steps, thermostat off in production (**Preparing the run**);
- the flags with their defaults and the two output files with their fields (the `md` command block and the file table);
- the three pass conditions and the k -frame drift rule (the acceptance table).

Follow the skills’ guide through design, plan, and test-first execution, approving each stage. Expected: red, then green, committed in that order.

(2) Add the reproduce rule:

💬 Add `week2/Makefile` with `make reproduce` that runs the crate in `week2/md/` in release mode with its default flags, writing to `week2/artifacts/`. Use `--manifest-path md/Cargo.toml` from `week2/`. Keep generated data and build files out of git. Commit.

The instructor’s gate runs `make reproduce` in your `week2/`, then recomputes the physics from the generated `artifacts/`.

④ VERIFY

Terminal: inside `week2/`.

(1) **The tests pass.**

➤ `cargo test --manifest-path md/Cargo.toml --release`

Expected: all tests pass.

- (2) **The tests check the physics and file output.** Find the test names in the terminal output, then search for those names in your editor to open their definitions. Confirm that the test code checks:
- the forces on all atoms sum to zero within numerical tolerance;
 - the potential energy is continuous at the cutoff;
 - the built program runs and produces readable `run.json` and `traj.jsonl` files (an *integration test* checks these parts working together).

The cutoff test must probe just inside r_c : checking only $r = r_c$ can miss a jump, because the code may return zero there even without shifting the potential.

- (3) **The default run passes the physics check.**

```
> make reproduce
cargo run --manifest-path md/Cargo.toml --release -- check artifacts
```

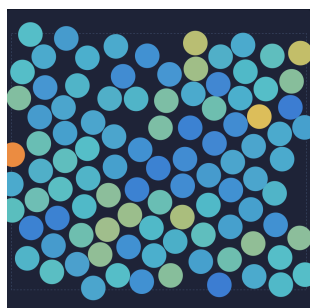
Expected: `md check` prints three measured values. They must satisfy `secular drift < 2e-3`, `|T_speed - 0.5| < 0.05`, and `chi2/dof < 2`, followed by `PASS`. Open `week2/md/src/main.rs`, find the `check` subcommand, and follow its calls to the energy calculation. Confirm it recalculates energy from the saved positions and velocities before comparing it with the stored energy fields.

- (4) **The video shows a fluid.**

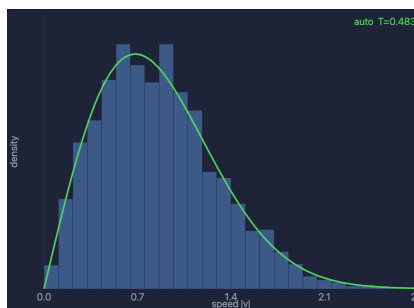
```
> cargo run --manifest-path md/Cargo.toml --release -- \
  video artifacts --out fluid.mp4
```

Open `week2/fluid.mp4`. Pass: the file is under 2 MB, the atoms drift without leaving the box, and the $g(r)$ panel settles into a liquid shape, one sharp first peak and a flat tail.

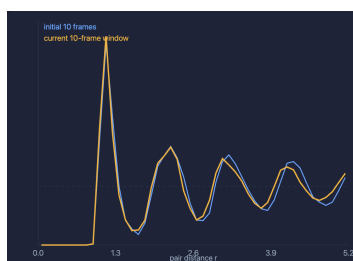
- (5) **The trajectory loads in the supplied viewer.** Open `week2-viewer.html`, drag `artifacts/traj.jsonl` and `run.json` onto it, and press play. Compare its four panels with the captures below, from the reference run with $N = 100$, $\rho = 0.8$, target $T = 0.5$, and seed 42, at the last of 200 saved frames. The speed and structure plots use recent frames; the histories use the full run. Your values will differ.



Positions: where are the atoms? Inspect their spacing and motion.



Speeds: does the measured histogram follow the predicted curve? A correct mean temperature alone does not establish the distribution's shape.



Structure: which neighbour shells persist? Read the peaks of $g(r)$ beyond the first shell.



Histories: is energy bounded? Temperature fluctuates while total energy should remain within a narrow band during unheated production.

Extension: Euler detonates. Add `--integrator euler` and run the contract with it. The dimer’s Euler error jumped at every close approach and reached 1.9 by $t = 5$; the dense liquid is far worse. Record when the relative energy error first exceeds 0.5, or when a state first becomes non-finite. Compare that time with the dimer and explain how close encounters change the stability requirement.

Part 5: Make the fluid faster, then heat it

Goal: a profiled release build with a cell list that reproduces the naive forces and a measured speedup, and a public $N = 400$ heating run whose viewer shows the distant $g(r)$ peaks disappearing as the crystal melts.

① READ

Scientific programs spend most of their time in a few lines. *Performance engineering* finds those lines by measurement and changes only them. In molecular dynamics the pair loop dominates, and its cost grows with the square of the atom count. This Part measures that cost, then replaces the pair loop with a cell list whose cost grows linearly, so a 400-atom heating run becomes affordable. That run shows melting, and Part 4’s $g(r)$ makes the loss of crystal order measurable.

Benchmark and profile. A *benchmark* measures how long a run takes. First compare the same simulation in two builds: `cargo build` produces a debug build with no compiler optimizations; `cargo build --release` enables them. Then find where the release build spends its time with a *profile*: a *sampling profiler* interrupts the program about a thousand times a second, records which function is running, and draws the totals.

To interpret these measurements, consider what Rust’s design already does. Part 3’s borrowed arrays let measurements read the state without copying it; its generic driver lets the compiler select the integrator before execution. Neither makes an all-pairs search stop being quadratic. When reading the profile, look for repeated requests for memory and `.clone()` calls, which copy these arrays. Distinguish compiler optimization, copying data, and searching fewer pairs when explaining a speedup.

Cell lists. With cutoff r_c , an atom interacts only with neighbours inside a circle of radius r_c , but an all-pairs loop still computes every pair distance. How can we skip distant pairs before calculating those distances? A *cell list* groups atoms by their current positions in rectangular cells with both sides at least r_c . Each cell can contain several atoms; these search regions are independent of the initial lattice. Any atom within the cutoff must lie in the atom’s cell or its eight neighbours, so only these nine cells need searching.

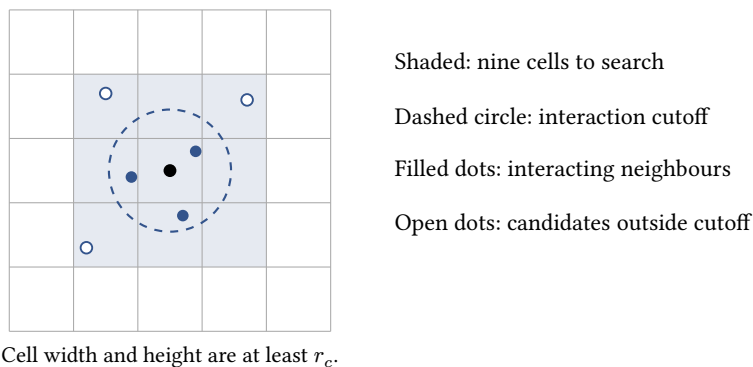


Figure 11: Candidate search around one atom, drawn schematically. Apply the exact distance and cutoff tests to every candidate. Wrap neighbour-cell indices at the box edges.

Use $n_x = \left\lfloor \frac{L_x}{r_c} \right\rfloor$ cells along x and cell width $\frac{L_x}{n_x}$; use the analogous rule along y . Visit each atom pair once. Deduplicate neighbour-cell indices when wrapping maps several offsets to the same cell, as in a two-cell-wide box. The shorter search must preserve the physical calculation. Compare its forces and energy with the all-pairs results within rounding tolerance on perturbed lattices, including pairs across a boundary, pairs at the cutoff, and this two-cell case.

Melting. A crystal melts when its atoms gain enough kinetic energy to exchange neighbours and lose long-range positional order. For the heating experiment, raise the target linearly from the starting temperature to a final value over production, rescaling velocities every 50 steps as in equilibration.

- Read [The Rust Performance Book’s profiling chapter](#). Explain how timing a run differs from finding where it spends time.
- Read the [sample README](#) through “Profiling Rust code.” Find how to record and inspect a profile.

② THINK

The goal is a program fast enough for a 400-atom heating run, with measurements that say why it is fast. The forces function visits every pair: $N(N-1)/2$ distance computations per step, 4950 for $N = 100$ and 79,800 for $N = 400$. The melting run needs 400 atoms and 2×10^4 steps, so the pair loop alone is 1.6×10^9 evaluations, which makes it the likely bottleneck. Benchmark to know whether you have a problem, and profile to know what to change: removing a function that takes five percent of the time saves at most five percent. The first change costs nothing, a release build; time both builds on your machine, since the ratio depends on the implementation and hardware. With the cutoff, most of those 79,800 distances belong to pairs that do not interact. In the cell-list figure, which dots can be skipped without changing the force, and why is visiting nine cells enough when every cell side is at least r_c ? At fixed density and cutoff, the candidates per atom stay bounded as N grows, so the work per step falls from order N^2 to order N . The cell list must reproduce the naive forces within rounding tolerance, so compare both force arrays on the same configurations before timing the optimized code. The fast program buys the heating experiment: 400 atoms on a lattice, with the temperature raised during production. Compare cold and hot pair structure with Part 4's $g(r)$, and decide which neighbour-shell peaks should survive heating before comparing your output with the reference in VERIFY.

The plan: measure first: time the NumPy baseline, the debug build, and the release build, then profile the release build to name the target. Build a cell list behind a flag that keeps the naive path, add a temperature ramp, and publish the page folder. Verify each by measurement: both force paths agree, the profile after the cell list, the speedup at three sizes, and the heating trajectory on the page.

③ DO

Measure, then build three features, then publish a page for them.

Terminal: inside `week2/` unless a step says otherwise.

- (1) **Install.** Make `md` a command you can run from any folder; the *PATH* is the list of folders the shell searches for commands:

Install `md` from `week2/md/` as a command on my *PATH*, built so that a sampling profiler can name its functions.

- (2) **Measure.** One measurement sits here rather than in VERIFY, because its result decides what to build next. Time the default run three ways, three times each: the NumPy baseline, your debug build, your release build.

```
> curl -fLO https://giggleliu.github.io/AMAT5315-2026Fall/downloads/week2-sim.py
python3 -m venv /tmp/venv && /tmp/venv/bin/pip install numpy
time /tmp/venv/bin/python week2-sim.py
cargo build --manifest-path md/Cargo.toml
time ./md/target/debug/md run --out /tmp/md-debug
time md run --out /tmp/md-release
```

NumPy uses compiled array operations, so do not assume which implementation wins. **Record your timings.** Add this template to `week2/README.md` and fill in the elapsed (wall-clock) time from three runs of each program. Sort the three times: the middle value is the median; the smallest and largest give the range.

Timing

Program	Median (s)	Range: min-max (s)
---	---	---
NumPy week2-sim.py	...	...-...
Rust debug	...	...-...
Rust release	...	...-...

Then find where the release build spends its time at $N = 400$:

```
> cargo install samplify --locked
samplify record md run --n 400 --eq-steps 200 --steps 1000 --out /tmp/md-prof
```

A browser tab opens the *Firefox Profiler*. In **Call Tree**, read the share of samples inside the force function and the elapsed time. Save the screenshot as `week2/profile-naive.png`. **Record your profile**. Add this template to `week2/README.md` and fill in the Naive row. Force share is the percentage of samples in the force function and its calls; elapsed time is the duration of the profiled run. Leave the Cell list row for verification after optimization.

Profile

Version	Force share (%)	Elapsed time (s)
---	---	---
Naive	...	...
Cell list	...	...

Commit the screenshot and README. The function with the largest share is the target of the next step.

- (3) **Optimize**. Brainstorm, plan, and build the cell list now that the profile has named the target. The naive path stays behind a flag so VERIFY can compare both:

💬 Brainstorm with me to speed up the md force calculation in `week2/md/` with a cell list. Request: keep the original force behind `--force naive` and make `--force cells` the default.

Before it starts, tell it, from READ's **Cell lists**:

- the cell rule: $n_x = \lfloor L_x/r_c \rfloor$ cells of width L_x/n_x along x , likewise along y , nine cells searched with wrapped and deduplicated indices;
- the equality cases: perturbed lattices with pairs across a boundary, pairs at the cutoff, and a two-cell-wide box, agreeing within rounding tolerance.

Follow the skills' guide through design, plan, and test-first execution, approving each stage. Expected: red, then green, committed in that order, with all earlier tests still passing.

- (4) **Heat**. A heating run rescales velocities during production toward a target that rises linearly with the production step. It gains energy on purpose, so keep it separate from the unheated `artifacts/` run.

💬 Add a `--ramp-to` flag to md run: during production, raise the thermostat's target temperature linearly from `--temperature` at step 0 to `--ramp-to` at the last step. Request: record `ramp_to` in `run.json`. Commit.

Reinstall so the copy on your PATH has both new features:

```
> cargo install --path md
```

- (5) **Publish**. Put a copy of the viewer in the repository's page folder and publish it with *GitHub Pages*, GitHub's free hosting of one folder of a repository as a website. Return from `week2/` to the repository root:

```
> cd .. && mkdir -p docs
  curl -fLo docs/index.html \
    https://giggleliu.github.io/AMAT5315-2026Fall/week2-viewer.html
```

💬 Publish `docs/` with GitHub Pages. Request: the page address under a Pages heading in `week2/README.md`. Commit and push, then show me the page.

This page **stays empty until** the heating run in VERIFY writes data.

④ VERIFY

Terminal: inside `week2/` until the last check. Each check confirms one feature by measurement and names the file or printed output the instructor inspects and the condition it must meet.

- (1) **Both force paths agree**. Run:

```
> cargo test --manifest-path md/Cargo.toml --release
```

Pass: the last line reads `test result: ok. N passed; 0 failed; ...`, and the list above it includes a test that both force paths give the same forces and energy, and a test of the heating schedule. If either is missing, ask the agent to add it.

- (2) **Release beats debug.** File: the **Timing** table in `week2/README.md`.

Pass: three rows, each a median and a range from three runs, and the release median below one third of the debug median. Which of NumPy and release Rust wins is recorded, not prescribed; the reference machine gave:

Program	Median (s)	Range (s)
NumPy <code>week2-sim.py</code>	10.86	10.71 to 10.86
debug <code>md run</code>	7.73	7.7 to 7.75
release <code>md run</code>	0.67	0.67 to 0.7

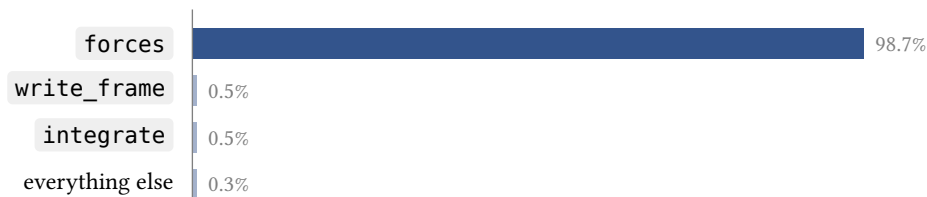
Timing: the contract run, $N = 100$, 12000 steps, three runs each.

- (3) **The profile names the hot spot.** Profile the cell-list build the same way; the reinstalled `md` uses cells by default:

```
> samplify record md run --n 400 --eq-steps 200 --steps 1000 --out /tmp/md-prof
```

Save the screenshot as `week2/profile-cells.png`. Fill in the Cell list row of the **Profile** table in `week2/README.md` with the force share and elapsed time.

Files: `week2/profile-naive.png`, `week2/profile-cells.png`, and the **Profile** table in the README. Pass: the naive force share is above 90%, and the cells elapsed time is below the naive elapsed time. The share after the cell list need not fall, since a percentage is a share of the whole run; the reference kept 95.8% in forces after the cell list.

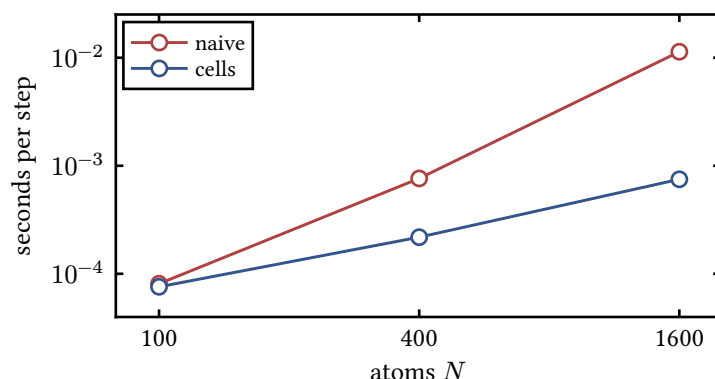


Where the time goes in the naive reference run, $N = 400$, timed per stage. Your Call Tree should show a force share of this order.

- (4) **Speedup grows with N .** Have the agent measure the scaling:

Benchmark `md run` with `--force naive` against `--force cells` at `--n 100`, `400`, and `1600`, with `--steps 500` `--eq-steps 100` and the defaults otherwise, three runs each. Request: a Benchmark table in `week2/README.md` with columns `N`, `naive`, `cells`, and `speedup = naive / cells`, each a median and range, and `week2/scaling.png` of seconds per step against `N` with the two lines labelled. Commit.

Files: the **Benchmark** table in the README and `week2/scaling.png`. Pass: six medians with ranges, a speedup column that rises with N and exceeds 2 at $N = 1600$, and two labelled lines in the figure that match the table. If the data do not resolve an advantage, extend the timed runs and inspect the profile; do not claim a speedup the table does not show. One README sentence explains the slopes using the pair-search diagram.



Scaling of the reference program, 600 steps, median of 3 runs, both axes logarithmic. Speedups 100: 1.1 $\times$, 400: 3.5 $\times$, 1600: 15.2 $\times$. Expect the same shape, not the same numbers.

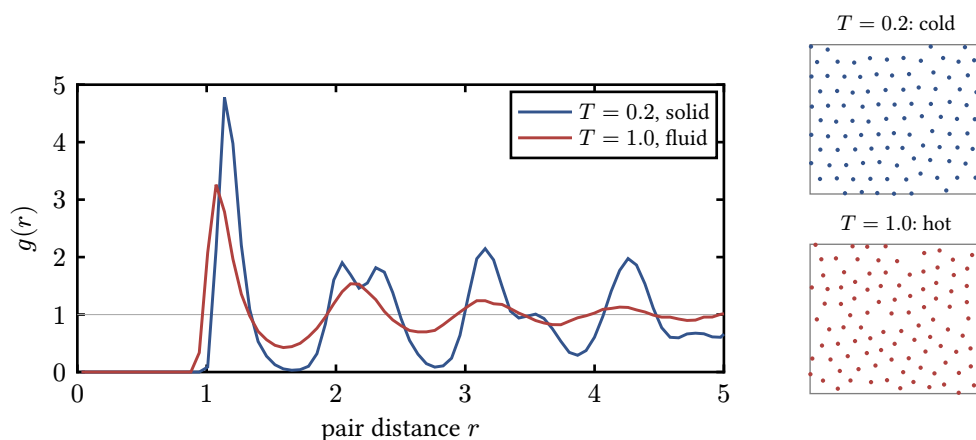
- (5) **Cold stays ordered, hot melts.** Return from `week2/` to the repository root. Run the heating trajectory into the page folder, then the two fixed-temperature runs the reference figure uses:

```
> cd ..
md run --n 400 --temperature 0.2 --ramp-to 1.2 \
  --steps 20000 --sample-every 100 --out docs
md run --temperature 0.2 --out /tmp/cold && md video /tmp/cold --out week2/cold.mp4
md run --temperature 1.0 --out /tmp/hot && md video /tmp/hot --out week2/hot.mp4
```

Then push the evidence:

Commit and push the heating run in `docs/`, the two videos in `week2/`, and the README. Request: the README lists the commands that reproduce every table and figure.

Files: `week2/cold.mp4` and `week2/hot.mp4`, plus the Pages address from the README opened in a private window. Pass: both files are under 2 MB; in `cold.mp4` the atoms vibrate about lattice sites and $g(r)$ keeps sharp peaks out to long range; in `hot.mp4` atoms exchange neighbours and $g(r)$ flattens to 1 beyond the first peak, as in the reference below. On the page, 400 atoms and 200 frames show without a login, the long-range contrast at the last frame is below the first frame's, and the temperature history rises from 0.2 to 1.2.



Cold and hot reference runs: right, final particle snapshots from the two $N = 100$ runs; left, their time-averaged $g(r)$, at density $\rho = 0.8$ and seed 42. Cold: sharp peaks at the lattice's neighbour shells, the first at $r \approx 1.14$, persisting to long range. Hot: only the first shell survives, since atoms still avoid overlap, and $g \rightarrow 1$ beyond it.

Extension: a sharper melting detector. The static structure factor $S(\vec{q}) = \left| \sum_j e^{i\vec{q} \cdot \vec{r}_j} \right|^2 / N$ is the Fourier-space fingerprint of order: sharp peaks in the solid, rings in the fluid. Compute it across your ramp run and identify where the peaks vanish. Compare with $g(r)$.

Further reading. This two-dimensional teaching model uses one pair potential and a velocity-rescaling thermostat. For broader molecular simulation, see Frenkel and Smit [1] or Allen and Tildesley [4]. Established packages include LAMMPS [5], GROMACS [6], HOOMD-blue [7] and OpenMM [8]. Inspect a package's force model, integrator and checks before trusting its results.

- [1] D. Frenkel and B. Smit, *Understanding Molecular Simulation: From Algorithms to Applications*, 3rd ed. Academic Press, 2023. doi: [10.1016/C2009-0-63921-0](https://doi.org/10.1016/C2009-0-63921-0).
- [2] E. Hairer, C. Lubich, and G. Wanner, "Geometric Numerical Integration Illustrated by the Störmer–Verlet Method," *Acta Numerica*, vol. 12, pp. 399–450, 2003, doi: [10.1017/S0962492902000144](https://doi.org/10.1017/S0962492902000144).
- [3] J. C. Maxwell, "Illustrations of the Dynamical Theory of Gases. Part I. On the Motions and Collisions of Perfectly Elastic Spheres," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 19, pp. 19–32, 1860, doi: [10.1080/14786446008642818](https://doi.org/10.1080/14786446008642818).
- [4] M. P. Allen and D. J. Tildesley, *Computer Simulation of Liquids*, 2nd ed. Oxford University Press, 2017. doi: [10.1093/oso/9780198803195.001.0001](https://doi.org/10.1093/oso/9780198803195.001.0001).
- [5] A. P. Thompson *et al.*, "LAMMPS: A Flexible Simulation Tool for Particle-Based Materials Modeling at the Atomic, Meso, and Continuum Scales," *Computer Physics Communications*, vol. 271, p. 108171, 2022, doi: [10.1016/j.cpc.2021.108171](https://doi.org/10.1016/j.cpc.2021.108171).
- [6] M. J. Abraham *et al.*, "GROMACS: High Performance Molecular Simulations through Multi-Level Parallelism from Laptops to Supercomputers," *SoftwareX*, pp. 19–25, 2015, doi: [10.1016/j.softx.2015.06.001](https://doi.org/10.1016/j.softx.2015.06.001).
- [7] J. A. Anderson, J. Glaser, and S. C. Glotzer, "HOOMD-blue: A Python Package for High-Performance Molecular Dynamics and Hard Particle Monte Carlo Simulations," *Computational Materials Science*, vol. 173, p. 109363, 2020, doi: [10.1016/j.commatsci.2019.109363](https://doi.org/10.1016/j.commatsci.2019.109363).
- [8] P. Eastman *et al.*, "OpenMM 7: Rapid Development of High Performance Algorithms for Molecular Dynamics," *PLOS Computational Biology*, vol. 13, no. 7, p. e1005659, 2017, doi: [10.1371/journal.pcbi.1005659](https://doi.org/10.1371/journal.pcbi.1005659).

Final verification: show the evidence from a fresh clone

On GitHub, check these paths in your `AMAT5315-2026Fall-Exercise` repository:

```
docs/
  index.html, traj.jsonl, run.json    ← published heating run
  superpowers/                      ← Part 3–5 designs and plans
week2/
  Makefile                          ← reproduce rule
  md/
    Cargo.toml, Cargo.lock          ← the Rust crate
    src/, examples/, tests/         ← simulator, both force paths, tests
  README.md                         ← Timing, Profile, Benchmark, Pages, recording
  REVIEW.md                         ← review findings, each fixed or not fixed
  field.png, dimer.png, scaling.png ← Part 2, 3, and 5 figures
  fluid.mp4, cold.mp4, hot.mp4      ← md video renders
  profile-*.png                     ← profiler screenshots
```

From the README, defend the force check, the Euler/Verlet comparison, the equilibrium speed law, and the evidence for faster simulation and loss of crystalline order. In your code, point to the owner of the particle arrays, explain a borrow, show how the trait lets the same experiment use either integrator, and identify one macro and its job.

Run the five checks. Check 3 uses the agent because it reviews code:

(1) **Tests pass from a fresh clone.** In a fresh terminal:

```
> cd "$(mktemp -d)"
gh repo clone "$(gh api user --jq .login)/AMAT5315-2026Fall-Exercise"
cd AMAT5315-2026Fall-Exercise/week2
cargo test --manifest-path md/Cargo.toml --release
```

Expected: all unit tests and the CLI integration test pass.

(2) **Physics recomputes.** In the same `week2/` terminal:

```
> make reproduce
cargo run --manifest-path md/Cargo.toml --release -- check artifacts
```

Expected: **PASS** with all three bounds in Part 4's acceptance table satisfied.

- (3) **Code survives review.** Start a fresh agent session at the clone's root:

Review week2/ against the designs and plans in docs/superpowers/. Look for wrong physics, weak tests, and claims the results do not support. Write your findings in week2/REVIEW.md.

Mark each finding **fixed** with its commit hash or **not fixed** with a concrete reason. After fixes, rerun tests and the physics check. Commit and push the review and fixes.

- (4) **The page explains melting.** Open the README's Pages URL in a private window. Expected: four synchronized views, **400 atoms**, and **200 frames**, with no login. Show the falling long-range contrast and disappearing distant $g(r)$ peaks as the temperature rises.
- (5) **The recording.** One take, at most two minutes, of your screen. Run check 2 in a fresh terminal until **PASS** appears. Then open the Pages address, pause the viewer near the cold start at $T \approx 0.2$ and again once $T \approx 1.0$, and say what the $g(r)$ plot shows at each: which peaks stand at 0.2 and which remain at 1.0. Attach the file to a GitHub release of the repository, not to git, and put its link in the README beside the Pages address.

Show all five checks to the instructor at the next session. Nothing is submitted.